NEMA: Automatic Integration of Large Network Management Databases

Fubao Wu, Han Hee Song, Jiangtao Yin, Lixin Gao, Mario Baldi, Narendra Anand

***Abstract*—Network management, whether for malfunction analysis, failure prediction, performance monitoring and improvement, generally involves large amounts of data from different sources. To effectively integrate and manage these sources, automatically finding semantic matches among their schemas or ontologies is crucial. Existing approaches on database matching mainly fall into two categories. One focuses on the schema-level matching based on schema properties such as field names, data types, constraints and schema structures. Network management databases contain massive tables (e.g., network products, incidents, security alert and logs) from different departments and groups with nonuniform field names and schema characteristics. It is not reliable to match them by those schema properties. The other category is based on the instance-level matching using general string similarity techniques, which are not applicable for the matching of large network management databases. In this paper, we develop a matching technique for large NEtwork MAnagement databases (NEMA) deploying instance-level matching for effective data integration and connection. We design matching metrics and scores for both numerical and non-numerical fields and propose algorithms for matching these fields. The effectiveness and efficiency of NEMA are evaluated by conducting experiments based on ground truth field pairs in large network management databases. Our measurement on large databases with 1,458 fields, each of which contains over 10 million records, reveals that the accuracies of NEMA on both numerical and non-numerical instances are up to 95%.**

***Index Terms*—Network management, Database Matching, Graph Database.**

I. Introduction

WITH the development of big data analytics and data mining techniques, scalable network measurement and analysis techniques have been used in finding hidden information or patterns to help with network management, network monitoring and network security [44], [12], [19], [30], [39], [38]. In one scenario of important network management, big network vendors serving various customers own a multitude of databases on network product information, network configurations, geographical information on deployment sites, incident logs, etc. Because the databases serve different departments, they are usually separated from each other and independently managed by different departments and groups. However, network data are inherently designed to host "connections" among different devices, groups that the devices are in, and functionality the devices serve together; they are often required to be shared and put together to be used in many important tasks such as network prediction, semantic query, network analysis and fault detection. For example, with the data connections, one can easily model a graph database to query related product configurations and performance issues around faulty devices or service incidents, which are matched with another product-related table [33], [1]. As many studies [11], [15], [8], [42] point out, the discovery of matching fields is the most crucial and foremost step for the integration of databases. For this reason, in this paper we aim to automatically construct such matchings that lead to efficient network management and analysis.

There is an abundance of research on field matching and integration approaches for different data formats in different contexts such as relational databases, XML and object-oriented data formats [17], [36], [43], [6]. Existing database matching approaches include two main categories of techniques. One is based on schema-level matching, which exploits metadata using schema characteristics such as field names, data types, structural properties and other schema information [14], [4], [18]. However, network management databases from different sources have different design standards and naming conventions [21], [10]. Even similar fields can have different names (e.g. "product_family" can also be named as "product series"). The other category is instance-level matching, which uses the record values of two fields to calculate the similarity and to determine matched fields [9], [34], [31]. Most of the previously proposed schemes rely on sampling or machine learning techniques that hope to extract common patterns from the matching data corpus. However, it is difficult to reliably construct dictionaries, corpus, and training labels with large datasets when the naming convention is not consistent as there are many different ways to name and abbreviate same fields.

The challenges for matching these network databases are:

1) The database design is not ideally uniform. The data tables are created in different groups and departments by different people. Therefore, it is not reliable to use field name similarity to match data directly and easily.
2) The data is noisy and irregular. Some table fields contain unexpected records such as null, invalid values and typos. Some table records are either partly missing, incomplete or incorrect. Some fields have a large amount of records, while some have very few records.
3) The table contents are complicated and heterogeneous with numerical and non-numerical data formats.
4) No thesauri or auxiliary information exist that we can rely on for matching. The only set of observation available is the database itself.

Fubao Wu and Lixin Gao are with the Department of Electrical and Computer Engineering, University of Massachusetts, Amherst. E-mail: fubaowu@umass.edu

Han Hee Song, Jiangtao Yin, Mario Baldi and Narendra Anand are with Cisco.

To solve these challenges, we propose an automatic matching technique for large NEtwork MAnagement databases (NEMA) to construct a graph database for network management and analysis. Specifically, we match numerical and non-numerical fields respectively. For numerical fields, we develop column-wise matching metrics including range difference similarity and bucket dot product similarity metrics as final matching scores to measure the correlation of fields. For non-numerical fields, we develop a record-wise matching metric "top priority match metric" to correlate fields. We also apply minHash and locality sensitive hashing algorithms to the network management databases for more scalable non-numerical matching. We implement and experimentally evaluate the effectiveness and efficiency of NEMA on large network management databases.

The rest of this paper is organized as follows. We define the problem in Section II. Section III describes NEMA matching algorithms in detail including both numerical matching and non-numerical matching. Experimental evaluation is presented in Section IV and related work is shown in Section V. We conclude our paper in Section VI.

II. Problem Description

Given structured network management databases, our goal is to create a graph database of network management by finding the most accurate matched field pairs among different tables in these databases. The matching of two fields is determined by the matching score measured by their record pair similarities. We utilize the matched results to construct a graph database for semantic query, network analysis, network prediction, etc. [29], [24], [25].

To illustrate our problem and algorithms clearly, we use three sample tables below as a toy example throughout the rest of this paper. In Table I, PRODUCT Table (T_P) contains 2 fields $product_id$ (primary key), and $family$ with 7 records respectively. In Table II, INCIDENT Table (T_I) contains 2 fields $incident_key$ (primary key), and $prod_key$ with 7 records respectively. In Table III, ORDER Table (T_O) contains 3 fields $order_key$ (primary key), $incident_id$, and $product_name$ with 7 records respectively. The problem is to find whether these 7 fields match among the Table T_P, T_I and T_O by evaluating the matching of their records, then to construct a graph database for efficient network analysis and management.

TABLE I: PRODUCT (T_P)

product_id	*family*
107	AIR series
108	con series
109	con series
150	47-7000
151	cisco0500
152	80-7066C
153	con5100

TABLE II: INCIDENT (T_I)

incident_key	*prod_key*
201	107
202	107
203	108
204	109
207	150
208	151
209	152

TABLE III: ORDER (T_O)

order_key	*incident_id*	*product_name*
301	201	AIR1212AC
302	201	AIR1002
303	203	con5122
304	204	mem-4700m-64d=
305	207	47-7066C
306	208	cisco0510
307	208	cs6012

Record Matching: Given two instances e1 and e2, a record matching function is defined as a 4-uple: $\langle e_1, e_2, v, r\rangle$ where e1 and e2 are two field records; v is a similarity score (typically in the [0, 1]) between e1 and e2; r is a relation (e.g., equivalence, part-of, overlapping, etc.) between e1 and e2. The matching function $\langle e_1, e_2, v, r\rangle$ asserts that the relation r holds between the record e1 and e2 with score v. For numerical matching, only if two records are equal, they are considered matched. For example, records {107, 108, 109, 150, 151, 152 } in $product_id$ field in table T_P are matched as the equal records in $prod_key$ field in table T_I, respectively. For a non-numerical pair, however, if the similarity score of two records are higher than a threshold based on a similarity metric, they are considered matched. Here we consider the part-of relation in the network management databases for meaningful relations including subgroup versus group, product versus product family, subseries versus series, and so on. For example, the record "con5122" in $product_name$ field and the record "con5100" in $family$ field can have high similarity score with part-of relation. A record pair which is matched is called a matched record pair, and it is called a non-matched record pair if the pair is not matched.

Field Matching: Given two fields f_1, f_2 and a threshold T, we define $sim(f_1, f_2)$ as the matching/similarity score (e.g. Jaccard similarity) between two fields f_1 and f_2. If $sim(f_1, f_2)$ value is above T, we call (f_1, f_2) a matched field pair, otherwise it is called a non-matched field pair. In the toy example, $sim(product_id, prod_key)$ can have a high matching score above a certain threshold, so $(product_id, prod_key)$ is correlated and matched. Moreover, the field pair $(family, product_name)$ can also be matched in terms of many matched record pairs such as some pairs $(AIRseries, AIR1002)$, $(47-7000, 47-7066C)$ and $(con5100, con5122)$, and so on.

Graph Database: One effective way to utilize matched results is to construct a graph database for semantic query, network analysis, network prediction, etc. [29], [24], [25], which is also our goal. We define a graph database as a labeled, attributed and undirected graph $G = (V, E, L_v, L_e)$ where V is the node set containing all the records appearing in the fields which match, E is the edge set between node pairs for node set V. L_v is a set of label information of node set V, which are the index attributes for the columns of the table. L_e is a set of label information of edge sets E, which are the relations of two records from two tables' column attributes. Figure 1 shows an example of a constructed graph database from parts of matched results of Tables T_P, T_I and T_O. For example, a specific node v "product_id: 107" in the table T_P has a node label l_v "PRODUCT" and "Family:AIR series". Node v is connected to the node "Incident_key:202" with a Product-incident relation, to the node "Incident_key: 201" with a Product-incident relation, and to the node "Family:AIR series" with a Product-family relation.

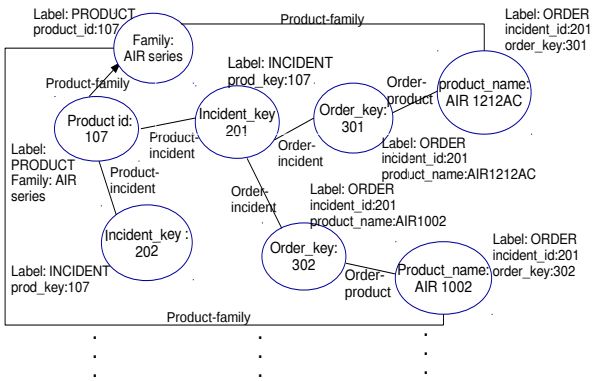

Fig. 1: Graph database created from Table T_P, T_I and T_O

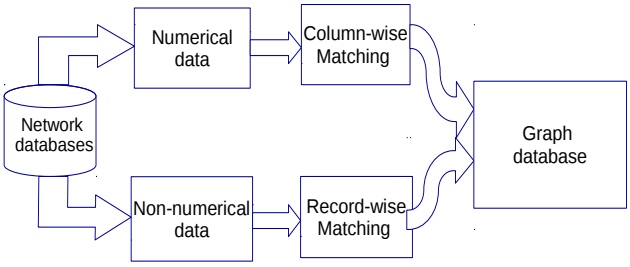

Fig. 2: System diagram of automatic integration for network databases

III. Matching Algorithm

To find whether two fields match, one simple way is to use field name matching. If the name of two fields are the same or similar, they are matched. However, this is not reliable for many database sources because they are noisy and irregular. Moreover, the network databases comprise of numerical and non-numerical fields, and they have different attributes and matching requirements. For numerical matching, we consider equivalence relation between record pairs. For non-numerical matching, however, we do not directly consider the equivalence relation as the matching standard. Because non-numerical record values are possible to be semantically correlated with different names. For example, the non-numerical fields $family$ and $product_name$ in the example table T_P and T_O have very few common characters on their names, but they are semantically correlated that $product_name$ has a part-of relation with $family$. Moreover, in terms of field records, records "cisco0510" and "cisco0500" in these two fields can be considered belonging to the same family and being matched with a high similarity. However, the record pair "47-7066C" and "80-7066C" are considered to be non-matched with different families, even though the two strings have many common characters. Hence, we make use of the record matching to decide whether two fields match to improve matching accuracy and satisfy semantic matching. Overall, we match numerical and non-numerical fields separately and design different matching algorithms for each of them.

A. System Overview

The system overview is shown in Figure 2. We divide the structured data into numerical data with only numerical fields and non-numerical data with only non-numerical fields. In each part, we develop an independent matching algorithm for field matching. Matching algorithms for numerical and non-numerical data are quite different, which will be introduced in section III-B and III-C. The results of each part are combined together to load into a graph database.

B. Numerical Matching

Numerical fields are table fields with records which are numerical values. For example, the $incident_key$ and $prod_key$ in Table T_I are numerical fields. Their record values construct a basis for similarity metrics of fields. We define each numerical field record values as a set. This is transfered to a problem of set similarity.

There are some common methods for solving set similarity including Jaccard index, Dice index, Hamming distance, cosine similarity [7], etc. However, it is not practical to just use one method to get accurate decision bounds of matching because of the noisiness and complexity of the structured databases. We propose a synthetic column-wise numerical matching algorithm to get the decision bounds to determine whether two fields are matched or not. The numerical matching algorithm is shown in Figure 3.

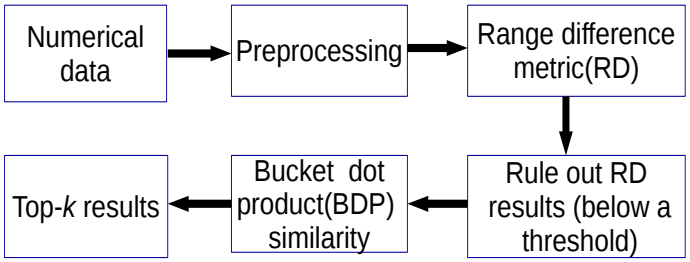

Fig. 3: Numerical matching flow

The process of this algorithm is shown as follows:

- We preprocess the numerical data, such as removing null values, negative values and some exceptional non-numerical values in every field.
- We apply range difference similarity metric to all the preprocessed numerical field pairs between every two tables.
- After we have got the range difference similarity score for each field pair, a threshold T_r (which will be discussed later) will be decided to cut the filtered results.
- Finally, bucket dot product similarity metric will be applied to the range difference similarity metric's filtered results. Then we sort the similarity scores of all the pairs to select the most correlated field pairs, that is, top-k result as matched pairs with scores above a certain threshold T_b will be selected to input to a graph database.

1) Preprocessing: To deal with the irregular and noisy data, we do some preprocessing of field record values before the formal similarity calculations. This includes removing null values, negative values, any non-numerical values and

considering unique values only. We do not consider negative values because they are useless and noisy data in the real databases. Almost all the table fields are about identifications, keys or numbers which can possibly be matched among them.

2) Range Difference Similarity Metric: Considering the noisy and sparse characteristics of data, Jaccard similarity coefficient as a similarity metric, which measures how many common values between two sets, is not ideal for differentiating some matched pairs and non-matched pairs. For example, for an non-matched field pair with limited number of records, the number of common values in these two fields might take a large portion and hence the Jaccard similarity is very high for them. Their distribution of range, however, can be quite different, which is probably not to be matched in most cases. Therefore, we propose range difference (RD) similarity metric to measure the distribution of these field pairs first. Using RD similarity metric first, we can effectively and efficiently prune lots of unwanted computations, which can also hugely reduce time consumption for further matching.

Given two field record sets, A and B, we calculate their different percentiles A_i, B_i (i indicates the 10th, 20th or 30th percentile and so on). The RD value D_i for each percentile i is

$$D_i = \frac{|A_i - B_i|}{|A_i + B_i|} \tag{1}$$

We use 20th plus 30th percentile as the low range coverage, and 80th plus 90th percentile as the high range coverage, to generally cover the distribution of field record values. Hence the RD similarity score for a field pair (A, B) is defined as:

$$RDS(A, B) = 1 - \frac{D_2 + D_3 + D_8 + D_9}{4} \tag{2}$$

To keep consistent with the general similarity metric and be convenient for comparisons, we use 1 minus the averaged RD value as the RD similarity score RDS. The metric based on this similarity score is called range difference similarity metric. Using this similarity metric, we can get the similar distribution for matched pairs. The value of RDS is in [0, 1]. The bigger the similarity score, the more correlated the pair. There are several cases about the similarity score considered here:

(1) If there are no overlaps between two field ranges, RDS would be as low as the minimum value 0.

(2) If two fields have similar distributions, RDS would be higher, up to 1.

(3) If two field ranges overlap at the head, tail or in the middle, RDS can fall into a middle value.

In our toy example, the matched pairs $(incident_key, incident_id)$ and $(product_id, prod_key)$ have RDS values as high as 0.996 and 0.999 respectively. In contrast, the pairs $(incident_key, product_id)$ and $(prod_key, incident_id)$ have no overlaps with RDS value 0, which are not matched pairs. The more correlated the field pairs are, the higher RD similarity score they have. Therefore, using a threshold T_r to filter results, we can almost rule out case (1) and part of case (3), then mainly consider case (2) to differentiate them further. To minimize the error of RD similarity score in the first step, we can use a conservative threshold close to the boundary to only filter out definite non-matched pairs, which will be discussed in the section IV.

3) Bucket Dot Product Similarity Metric: After we consider the distribution of field pairs with range difference similarity metric, we propose bucket dot product (BDP) similarity metric to further refine the filtered results of RD similarity metric. BDP similarity metric is to divide the whole concatenated ranges of two fields into different bucket/bins and compress each bucket as one point to calculate dot product similarity. The intuition behind this is that matched pairs generally have more common values than non-matched pairs. If we increase the bucket size up to a certain value to calculate dot product, it can make the similarities of all the non-matched pairs decrease more, and meanwhile make the similarities of all the matched pairs drop less, thus it effectively increases the similarity gaps between matched pairs and non-matched pairs. Therefore, a good design of BDP will help significantly differentiate between matched pairs and non-matched pairs.

The general dot product similarity of two vectors X and Y with n elements is defined as follows:

$$dot(X, Y) = \sum_{i=1}^{n} X_i \cdot Y_i \tag{3}$$

We use the bucket number (b_n) to determine the number of buckets for calculating the dot product. Given two field record sets A and B, we first derive the required vectors A_v and B_v for the input to the BDP similarity calculation. The vectors A_v and B_v derived from A and B are constructed in this way. Given two sets A and B, we concatenate A and B's value ranges as a combined set C, and then divide C into several buckets according to the b_n. If there is any one value in A or B falling in a bucket, the bucket point for A_v or B_v' is 1, otherwise it is 0. Then we apply the general dot product similarity to A_v and B_v. Therefore, the BDP similarity (normalized) is defined as follows:

$$btDot(A_v, B_v) = \frac{\sum_{i=1}^{b_n} A_{vi} B_{vi}}{|A_v||B_v|} \tag{4}$$

where b_n decides the sparsity/density of range distributions. Because sets A and B usually have different sizes with different ranges, it would make sense for b_n the same for each set. For example, we calculate the BDP similarity for a field pair $(incident_key, incident_id)$ in Table T_I and T_O as A and B with $b_n = 3$. We first concatenate these two field ranges into a set {201, 202, 203, 204, 207, 208, 209}. Then we construct a set C ={{201, 202, 203},{204, 207, 208}, {209}} according to the bucket number b_n. After that, we get the vector A_v = {1, 1, 1} and B_v = {1, 1, 0}. Finally, the BDP similarity score is 0.816, which is high for matching. If we set b_n as 4, BDP similarity score for this pair is 1, which is the highest for matching.

b_n is also an important factor to affect the quality of this metrics. According to our experimental observations, it is affected by the data range and distribution. Generally, matched pairs would have more similar ranges than non-matched pairs. A trade-off value of b_n would effectively improve matched pairs' similarities more and also do not help improve non-matched pairs' similarities much, which can potentially increase more gaps between matched and non-matched pairs. The selection of b_n will be discussed later in the section IV.

4) Primary Key Constraint Matching: If we consider all numerical fields and then do match computations between each other, it would be very time-consuming and not scalable when we have a large number of fields. If there are n tables from database sources, and an average of m fields in each table, the maximum $\binom{n}{2} * m^2$ comparisons are required. If we consider only table A and table B matching in a semantic way, the primary key constraint [4], [20] can be utilized to reduce the time for pair comparisons. Moreover, relational databases generally have primary keys and closely-related foreign keys in a table, which describe the semantic meanings of the table. It is important to match two tables in this semantic way to construct an effective graph database. Assuming every table has at least a primary key, we can identify the primary key and use the primary key's records to compare with all other keys records in other tables. We identify the primary key as a field that has unique record values before we apply RD and BDP similarity metrics. For example, $product_id$ and $incident_key$ are identified as primary keys in these two tables T_P and T_I, and thus we only need to compare $(product_id, incident_key)$, $(product_id, prod_key$, $(incident_key, incident_id)$ and $(product_id, incident_id)$ in total, reducing 33% times of comparisons.

5) Final Top-k Selection: After we finish calculating the similarities of all the needed pairs, it is necessary to select top similarity pairs as the final results to construct a high-quality graph database for a graph processing system because the field pairs with higher similarity scores are more correlated and more accurate. To ensure the effectiveness of a graph database, we sort all the candidate pairs by the similarity scores in a non-ascending order, then we verify this final field pair matching results to select top-k field pairs, which involves only a little human labor.

C. Non-numerical Matching based on Top Priority Matching

We propose fast algorithms for non-numerical field matching–top priority match metric (TPM) for fast filtering, and match ratio score for final similarity computations. The diagram is shown in Figure 4. The main process of our non-numerical matching algorithms is as follows:

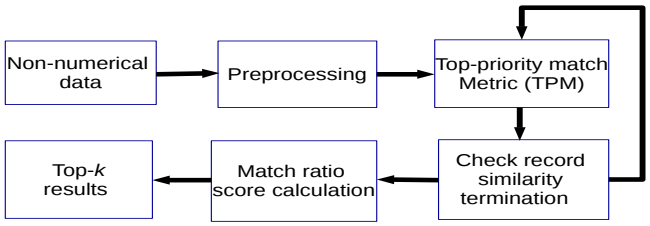

Fig. 4: Non-numerical matching flow based on TPM

- Preprocess non-numerical data: this is an important step that decides the quality of our non-numerical matching algorithm. After splitting non-numerical data from the original databases, we use our designed natural language processing methods of segmentation, stemming and prefixing for every field record value.
- Calculate the record-wise similarities iteratively: It is very time-consuming to apply cosine similarity to the combination of every record pair in a field pair. Considering the scalability of large-scale data matching, we propose top priority match metric for record-wise similarity calculation. In the process of iterative computations, we check the termination condition to terminate the iterations earlier, which significantly reduces time complexity.
- Calculate match ratio score: after record-wise similarity computations for every field pair are finished, we calculate the defined matching ratio score for each field pair.
- Select top-k results: we sort the field pairs by the matching ratio score in a non-ascending order, and the top-k field pairs are selected as the final results as an input into a graph database.

Here, we discuss each proposed steps in detail.

1) Preprocessing: Non-numerical matching considers partial match between two strings. For example, product "cisco0510" and product "cisco0500" are in the same series, which is considered as a partial match. Hence, we propose the following preprocessing method.

(1) Parse every record string A, remove null value, separate alphabetic and numerical characters into different new substrings, and tokenize the string words.

(2) Stem the alphabetic strings of the original record and new substrings.

(3) Reserve the prefixes with a certain length of the original numerical strings and the new substrings if they are digital substrings. The prefix length is 2 here according to the general characteristics of network databases and our experiments.

Each record string is preprocessed in those three steps above. For example, we have an original field record R {'mem-4700m-64d='} in Table T_O. We can obtain a string collection X {'4700', '64', '4700m', 'd', 'm', 'mem 4700m 64d', '64d', 'mem', '47xx'} after preprocessing R.

2) Top Priority Match Metric for Record-wise Similarity: One intuitive way is to preprocess all the combination of record pair comparisons and calculate the similarity of each record-wise pairs. That would be very time-consuming or even unfeasible when the data are large. Specially, if two fields are not correlated as a matched pair, it would be costly for useless computations. Therefore, we propose a fast record-wise matching algorithm called top priority match metric for record-wise similarity (TPM) to fulfill this. Intuitively, if two fields A and B are correlated, there will be a high percentage of record-wise pairs that have higher similarities. The probability of a matched record pair encountered is higher than non-matched record pairs. Therefore, we first sort all the preprocessed records in each two fields A and B, then we compute how many of records in A are matched with records in B from top to bottom, and vice-versa. The comparisons can hence be terminated as long as the current record pair similarity achieves below the similarity threshold T_{rn} we set for deciding the matching of a record pair, which greatly reduce the times of comparisons with combinations.

3) Record Pair Similarity: In this fast record-wise comparisons, the record pair similarity used is cosine similarity between two record collections after preprocessing two

records. It decides how and when to reduce the comparisons of matching in a fast and effective way. A threshold T_{rn} is to decide how similar a record pair is as a matched record pair for the network databases, which can also be adjusted by users.

Given a preprocessed string collection X and another preprocessed string collection Y, we remove duplicated elements and transfer them into a set XY(X union Y). Next we generate a binary vector V_x and a binary vector V_y according to the value distribution of X and Y in XY, then we calculate the cosine similarity $sim(V_x, V_y)$ between V_x and V_y as follows by getting their dot product divided by their magnitude multiplication.

$$sim(V_x, V_y) = \frac{V_x \cdot V_y}{|V_x|\,|V_y|} \tag{5}$$

For example, we have a preprocessed string collection X {cisco, 0510, cisco0510, 05xx}, and Y {cisco, 05xx, 0500, cisco0500}, we transfer them into a set of X union Y, XY {cisco, 0510, cisco0510, 05xx, 0500, cisco0500}. Then the binary vectors generated according to X, Y and XY are V_x {1, 1, 1, 1, 0, 0} and V_y {1, 0, 0, 1, 1, 1}. Finally, we calculate the cosine similarity of V_x and V_y as the similarity of X and Y, that is, $sim(X, Y) = (1 + 1)/(2 * 2) = 0.5$.

4) Matching Ratio Score: Matching ratio score is proposed to calculate the final similarity for a field pair. After we have gone through the reducing comparisons for record-wise similarities, we select the number of record pairs that have similarity scores above T_{rn}. A matching ratio score as a final field pair similarity is the average of ratios of top matched record pairs calculated as follows:

Given two non-numerical sets A and B, there are m items $\{a_1, a_2, ..., a_m\}$ in A and n items $\{b_1, b_2, ..., b_n\}$ in B. The matching ratio score(MR) between A and B is defined as follows.

$$MR(A, B) = \frac{1}{2} * (\frac{\sum_{i=1}^{m} A_i}{m} + \frac{\sum_{j=1}^{n} B_j}{n}) \tag{6}$$

where

$$A_i = \begin{cases} 1 & \text{if } \exists\ b_j \in B,\ sim(a_i, b_j) >= T_{rn} \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

and

$$B_j = \begin{cases} 1 & \text{if } \exists\ a_i \in A,\ sim(b_j, a_i) >= T_{rn} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

where $sim(a_i, b_j)$ and $sim(b_j, a_i)$ are the cosine similarities of the record pairs (a_i, b_j) and (b_j, a_i), respectively. MR value is in [0, 1] and it is the final similarity score to decide the correlation of each field pair in this record-level matching.

5) Final Top-k Results: After we calculate all the matching ratio scores of field pairs, we obtain a list of pairs sorted by the matching ratio scores. Similar to numerical field matching, matched non-numerical field pairs in the result list are more meaningful and important than non-matched field pairs, and thus we select top-k results of non-numerical field pairs with high matching ratio scores as an input to a graph database. K value can be selected by users for deciding most effective field pairs in a graph database and limiting the size of the graph database.

D. More Efficient Hashing Algorithm for Non-numerical Matching

The proposed TPM metric can be effective to distinguish between matched and non-matched non-numerical fields. However, it possibly involves all the pairwise record combinations in the worst time complexity, which is time-consuming for large databases with millions of records. Therefore, we propose applying more scalable minHash-locality sensitive hashing algorithm (MH-LSH) [23] to estimate the matching score of non-numerical fields in the databases. It can greatly reduce the comparison size and time for non-numerical record-wise pairs with little cost of matching accuracy going down.

The proposed diagram for non-numerical matching based on MH-LSH is shown in Figure 5. The main process of the algorithm is as follows:

- Preprocess non-numerical data: this step is the same as the preprocessing step in the non-numerical algorithm based on TPM to preprocess each record in a field.
- Select matched field pairs fast: we apply the locality sensitive hashing technique in the database field matching for fast selecting field pairs that are correlated. In this process, it uses minHash similarity to decide the filtering results for field pairs.
- Field pair similarity calculation: We use minHash technique in the databases to estimate the matching score of field pairs.
- Select top-k results: Same as the operation for non-numerical matching based on TPM, we sort the field pairs by the estimated matching score in a non-ascending order, the top-k field pairs are selected as the final results of field pairs as an input to a graph database.

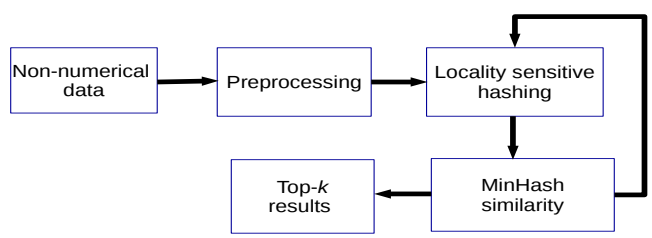

Fig. 5: Non-numerical matching flow based on MH-LSH

Here, we discuss important steps in detail.

1) Matching Score Estimation with MinHash: The similarity score of matched field pairs can be estimated fast with minHash signatures. Given two fields A and B, we can evaluate the similarity between them as follows. We choose n hash functions $h_1, h_2, ..., h_n$. For each hash function h_j, we let a signature of set A be $sgn(A) = \min_{i:a_i \in A} h_j(a_i)$ for $j \in n$. Let a signature of set B be $sgn(B) = \min_{i:b_i \in B} h_j(b_i)$ for $j \in n$. Then, the probability that the two sets have the same minHash signatures can be used to estimate the similarity between them.

$$\text{MHSim}(A, B) = P\{sgn(A) = sgn(B)\} \tag{9}$$

Here each record in a field A or B is also preprocessed with the same preprocessing method shown in section III-C1. The preprocessed record strings are combined into a new set like a word set in a document. We also use k shingle (a substring of length k) to create a set of k-shingles strings and apply n different hash functions on the set of strings.

2) Field Pair Selection with Locality Sensitive Hashing: Performing pairwise similarity measurement can be time consuming with large amounts of field pairs available. In order to identify which field pairs are similar quickly, we propose using locality sensitive hashing (LSH) to select the candidate field pairs.

The values of minHash signature $sgn(A)$ for one field A are grouped into b-tuples (referred to as sketches) with r rows. Similar field pairs have similar minHash signatures and hence have a high probability of having the same sketches. Moreover, dissimilar pairs have low chance of falling into the same sketch. The probability that two fields of A and B have at least one sketch (of size b) in common out of r is

$$P_C(A, B) = 1 - (1 - \mathrm{MHSim}(A, B)^b)^r \tag{10}$$

Therefore, we can find the candidate pairs with the designed number b and r. The selection of b and r is generally decided by a threshold $t = (1/b)^{1/r}$ shown in [23], which indicates how similar the two fields is to be considered as a candidate pair, and can also be set by users. In this way, if pairs with similarity above $P_C(A, B)$, they will be selected as candidate pairs to be further estimated, and the matching score between them with minHash will be calculated.

IV. Experimental Evaluation

In the section, we evaluate our technique NEMA for structured network database matching. To be specific, we measure the effectiveness of NEMA using ground truths for numerical and non-numerical data that are annotated by humans. Meanwhile, experiments on a large dataset are also conducted, and we show the top-k effective results of matching field pairs. Moreover, comparisons of NEMA with other based line methods are also shown.

A. Data Sets

The structured network management databases available in the form of database tables are provided by Cisco Systems, Inc. The data sets include "install_base" and "service_request" databases. They are generated by different departments and groups of the corporation, which are heterogeneous and diversely distributed.

In these databases, there are 21 tables which contain 1,458 columns. Each column has 10 million records on the average. Out of them, there are 679 numerical fields and 779 non-numerical fields. Therefore, a complete match involves the maximum 1,067,882 field matching decisions. With primary key constrains in numerical matching, there are 5 "primary keys" on average in each table, which would reduce to 374,326 field pairs matching.

We have ground truth field pairs that are annotated by humans to be matched or non-matched field pairs for a subset of the data. There are 60 field pairs of ground truth in numerical data and 40 pairs in non-numerical data.

For future reference, table names in service_request database start with "T_", and start with "X_" in install_base database, respectively.

B. Experimental Setup

We implement NEMA and its prototype system in the Python language. To evaluate the effectiveness of NEMA , we evaluate the numerical and non-numerical algorithm parts, respectively. For each part, we first evaluate its algorithms based on the ground truth data. Then all the column pairs in the large data sets are evaluated in the following experiments shown in sections IV-C2 and IV-D2, which shows the effectiveness of NEMA. Finally, we compare with the common matching system COMA [3] on the ground truth for both schema-level and instance-level matching.

To construct an effective graph database, identifying accurately matched pairs and reducing non-matched pairs appearing at the top are much more important than finding more non-matched pairs. Therefore, we use an overall metric "accuracy" (ACC) to evaluate our field matching algorithms. ACC is calculated with true positive(TP), true negative(TN), total positive pairs(P) and total negative pairs(N) (Here positive means matched and negative means non-matched).

$$ACC = \frac{TP + TN}{P + N} \tag{11}$$

C. Evaluation based on Numerical Data

We evaluate our technique NEMA on the numerical data in two parts. We use numerical ground truth data to evaluate the effectiveness of NEMA numerical algorithm. Then the matching results of whole numerical field pairs are also described.

1) Evaluating of Ground Truth: We show the evaluation result of NEMA numerical algorithms and the compared baseline method-Jaccard similarity using numerical ground truth. In the data set, there are 30 matched ground truth field pairs which are originally from fields pairs annotated by humans or from join operations in the databases and proved to be matched field pairs. We randomly sample some field pairs and also select 30 non-matched field pairs checked by humans.

Table IV shows 10 matched and 10 non-matched field pair examples of numerical ground truths. Columns "Table.field A" or "Table.field B" shows a table name and its field pair name to be compared. The matched class column indicates that the field pair is matched with value "1" or non-matched with value "0". For example, in the first row, "T_INCI.prod_hw_key" indicates a field "prod_hw_key" in the table "T_INCI", and "T_HW_PROD.bl_prod_key" indicates a field "bl_prod_key" in the table "T_HW_PROD". This field pair about products' key is matched, which is indicated with "1" in the matched class value. The remaining rows share the same characteristics as well.

The problem of similarity of a numerical field pair is modeled as the problem of similarity of a set pair. The baseline

TABLE IV: Example of numerical ground truth field pairs

No.	Table.fieldA	Table.fieldB	Matched class
1	T_INCI.prod_hw_key	T_HW_PROD.bl_prod_key	1
2	T_INCI.cur_ct_key	T_CT.bl_ct_key	1
3	T_INCI.up_tech_key	T_TECH.bl_tech_key	1
4	T_INCI.inci_id	T_INCI_I2.inci_id	1
5	T_INCI.bl_cot_key	T_COT.bl_cot_key	1
6	T_INCI.inci_id	T_OR_HD.inci_id	1
7	T_INCI.item_id	T_PROD.item_id	1
8	T_INCI.ins_site_key	T_SITE.bl_site_key	1
9	T_OR_LN.prod_key	T_PROD.bl_prod_key	1
10	T_OR_HD.header_id	T_OR_LN.header_id	1
1	T_OR_HD.order_dur	T_OR_LN.loc_key	0
2	T_CAL.bl_cal_key	T_PROD.item_id	0
3	T_INCI_I2	T_DEFT.deft_id	0
4	T_INCI_I2.res_time	T_TECH.sub_tech_id	0
5	X_PRO.list_price	T_TECH.sub_tech_id	0
6	T_INCI_I2.res_time	T_TECH.bl_tech_key	0
7	T_OR_HD.deliv_dur	T_DEFT.deft_key	0
8	T_OR_LN.hold_dur	T_TECH.sub_tech_id	0
9	T_IN.serlevel_key	T_INCI.last_dur	0
10	T_IN.closect_key	T_INCI.resp_tz	0

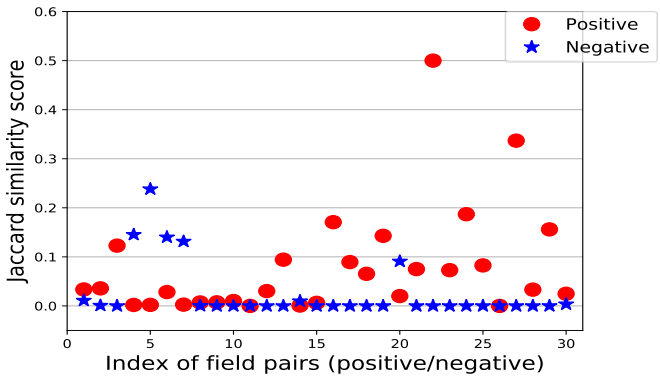

Fig. 6: Jaccard similarity metric on the numerical ground truths

method for the similarity of a numerical field pair is the well-known Jaccard similarity which measures the similarity of two given sets.

Figure 6 shows the Jaccard similarity scores on these ground truth field pairs. Red circle represents matched pairs and blue star represents non-matched pairs. X axis denotes the index of these 30 matched and 30 non-matched field pairs, and Y axis indicates Jaccard similarity score. From this figure, we can see that there are about half of positive and negative pairs mixed together from which are difficult to differentiate.

Figure 7 shows the RD similarity metric on these ground truth field pairs. We know that a pair is more likely to match if its range difference similarity score is larger. There are about only 30% positive and negative pairs mixed together, which has greatly improved the result over the Jaccard similarity result.

From the previous result of RD similarity metric, we can set a low threshold T_r and then rule out the result pairs below T_r as non-matched field pairs. To minimize the error of initial matching, we can select a conservative threshold T_r close to 0.1 (all of the uncertain pairs are above 0.1) to eliminate some generally precise non-matched pairs.

After we select the filtered field pairs from RD similarity metric, there are totally 39 field pairs left, that is, about 35% of

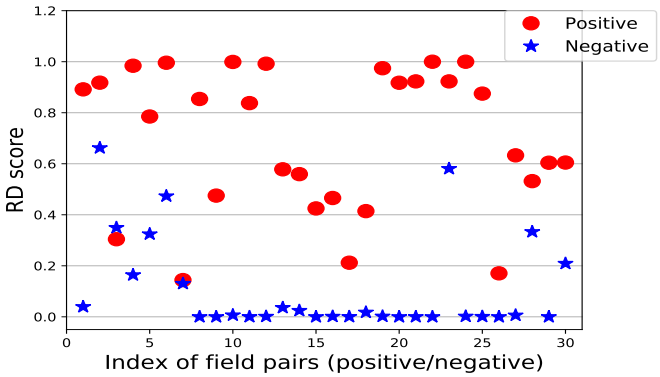

Fig. 7: RD similarity metric on the numerical ground truths

all ground truth pairs (all of them are non-matched field pairs) are pruned. In these 39 pairs, there are 30 matched pairs and 9 non-matched pairs waiting to be input to the BDP similarity metric.

Figure 8 shows the BDP result after the previous RD similarity metric result. X axis denotes the index of these 30 matched and 9 non-matched pairs, and Y axis indicates the normalized BDP similarity score in a non-ascending order. We can see from that the BDP result has very good decision line between matched pairs and non-matched pairs in which the final threshold T_b is chosen around 0.1. The final accuracy can be up to 95% when T_b is 0.1.

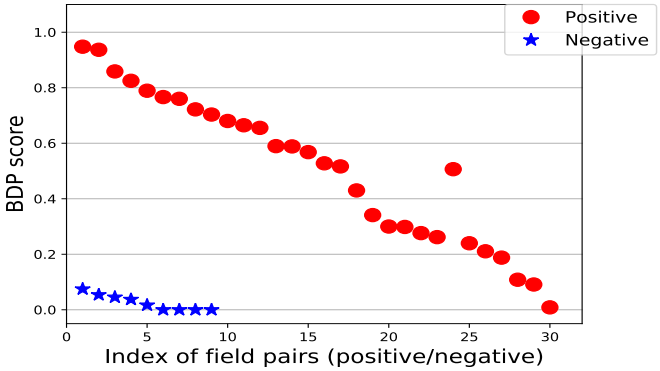

Fig. 8: BDP similarity metric on the remaining numerical ground truth pairs

The bucket number b_n is the main factor affecting the BDP similarity and the final results. To evaluate the effectiveness of different b_n values, we use all the numerical ground truth to test the accuracy with different b_n values from 100, 200, 300, 400, 500 up to 100,000,000 where there exists some numerical records that have values up to 10 billion in almost all the fields. Figure 9 shows the accuracy of BDP similarity metric with different b_n values when the threshold T_b is set at 0.1 and 0.2. It shows a similar summit that the accuracy is at an optimal value when the b_n is around from 10,000 to 200,000. BDP accuracy goes down when the b_n becomes smaller or bigger. We use a trade-off value $b_n = 50,000$ in our experiments to avoid overfitting.

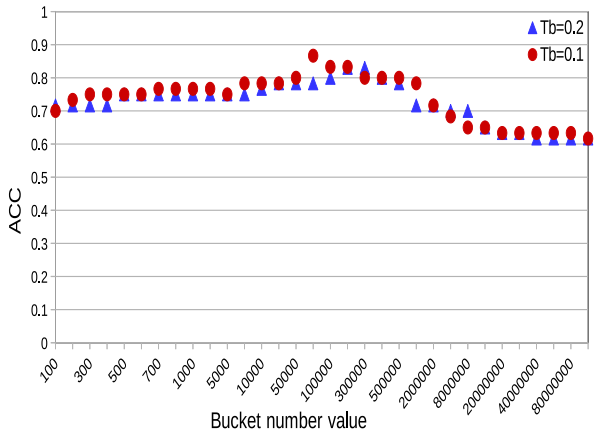

Fig. 9: BDP accuracy comparison with different bucket numbers on the numerical ground truths

2) Top-20 Similarity Results: The matching experiment based on all the 679 numerical fields is shown here. Table V shows top-20 matching results. We use RD similarity score threshold $T_r = 0.1$ and bucket number $b_n = 50,000$ for BDP similarity computation. All the rows are accurate matches, which are also confirmed by humans. The accuracy can be up to 100% for the top-20 results, which shows a great capacity and potential for our numerical matching algorithm applied on large data sets, also reducing lots of human labor for matching. Moreover, with our system-aided matching findings, we can find some pairs matching which are difficult to be found with human annotations such as "changewg_key" and "subregion_key", indicating which regions that the workgroup mainly serves.

To ensure a graph database more meaningful and complete, the selection of top-k results is finally decided by users. Users can observe the top-k results and rule out the unwanted matches as the final input pairs to a graph database so as to introduce less "noisy" connections of the graph database.

TABLE V: Top-20 similarity result of numerical field pairs

Table.field A	Table.field B	BDP similarity
T_DEFT.deft_key	T_INCI_DE.bl_def_key	0.844
T_OR_LN.item_id	X_PRO.item_id	0.698
T_DEFT.deft_id	T_INCI_DE.defect_id	0.682
T_INCI.item_id	X_PRO.item_id	0.673
T_DEFT.deft_key	T_PROD.item_id	0.64
X_INS.item_id	X_PRO.item_id	0.597
T_PROD.bl_prod_key	**T_SUR.task_key**	**0.567**
T_INCI.changewg_key	T_WK.subregion_key	0.551
T_INCI.changewg_key	T_WK.wkgrp_key	0.551
T_INCI.changewg_key	T_WK.theater_key	0.551
T_INCI.currentwg_key	T_WK.theater_key	0.545
T_INCI.currentwg_key	T_WK.subregion_key	0.545
T_INCI.currentwg_key	T_WK.wkgrp_key	0.545
T_INCI.hwversion_id	T_PROD.bl_prod_key	0.507
T_INCI.createwkgrp_key	T_WK.theater_key	0.507
T_INCI.createwkgrp_key	T_WK.subregion_key	0.507
T_INCI.createwkgrp_key	T_WK.wkgrp_key	0.507
T_PROD.item_id	X_PRO.item_id	0.468
T_INCI.prod_hw_key	T_HW_PROD.bl_prod_key	0.463
T_SUR.evalwkgrp_key	T_WK.wkgrp_key	0.433

D. Evaluation based on non-numerical data

We evaluate our NEMA non-numerical algorithms based on TPM and Hashing on the non-numerical data in two parts as well. We use non-numerical ground truth data to evaluate the effectiveness of our algorithms. The matching results of whole non-numerical field pairs are then described.

1) Evaluating of ground truth: The experiment for evaluating of non-numerical ground truths is shown in this section. It achieves high accuracy of matching results. There are 20 matched ground truth pairs which are annotated by humans. Additionally, 20 non-matched ground truth pairs are randomly selected and verified. Part of examples are both shown in table VI.

TABLE VI: Examples of non-numerical ground truths

No.	Table.field A	Table.field B	Matching class
1	T_PROD.item_name	X_INS.item_name	1
2	T_COT.cpr_country	T_SITE.country	1
3	T_CT.temp_desc	T_PROD.item_desc	1
4	T_CT.ctserv_line	X_SAH.servline_name	1
5	T_INCI.curr_wg_name	T_WK.wkgp_name	1
6	T_CT.temp_desc	X_SAH.temp_name	1
7	T_SITE.cust_state	X_SAH.billto_state	1
8	T_CT.temp_name	X_SAH.temp_desc	1
9	T_PROD.prod_family	T_HW_PROD.family	1
10	T_PROD.prod_family	T_HW_PROD.erp_family	1
1	T_INCI.init_gp_name	T_SITE.address	0
2	T_DEFT.deft_submitter	T_SITE.email_addr	0
3	T_COT.cpr_country	T_INCI.summary	0
4	T_SITE.address1	X_PRO.prod_family	0
5	T_PROD.prod_family	X_SAH.hdrcust_name	0
6	T_INCI.tacpica_ct	T_HW_PROD.family	0
7	T_WK.wkgp_desc	X_PRO.physisn_loc	0
8	T_SITE.county	T_SUR.batchcot_name	0
9	T_INCI.customersw_ver	T_SITE.state	0
10	T_OR_LN.partsloc_code	X_INS.item_name	0

We first analyze the ground truth record-pairs and show the viability for the record pair similarity threshold T_{rn}. Table VII shows the record pair similarity scores of 9 different record pairs in a field pair ("T_PROD.prod_subgrp", "T_HW_PROD.platform"). The first 7 rows of pairs are matched record pairs that have higher similarity scores. The last 2 rows are not matched record pairs with lower score of 0.3333, which is lower than 0.4. It is conservative to capture the matching among different records with a similarity value 0.4 assigned to the threshold T_{rn}.

TABLE VII: Sample of non-numerical record pairs

T_PROD.prod_subgrp	T_HW_PROD.platform	Record similarity
c900 series	c900 series	1
c2950 series	c2916 series	0.8
1601r series	1601 series	0.775
css2950	css2916	0.667
C2960	C2960CX	0.577
C3560CX	C3560X	0.5
AIR35CE	AIR35SE	0.4
ts900	**cs900**	**0.333**
c800	**s800**	**0.333**

We demonstrate the effectiveness of non-numerical algorithms based on TPM and MH-LSH by calculating matching ratio score based on TPM and estimated matching score based on MH-LSH on non-numerical ground truths. Figure 10 shows

matching ratio scores of this ground truth data matching in a non-ascending order based on TPM. The matching ratio scores of almost all the matched pairs are above the non-matched pairs's. If we use the threshold 0.1 or select top-20 results from this, the accuracy can achieve about 95%, which shows the effectiveness of NEMA based on TPM. Figure 11 shows the matching scores of these ground truths in a non-ascending order based on MH-LSH. Although the decision boundary is not as good as the TPM-based result in Figure 10, the accuracy can achieve 90% when the threshold is 0.1 or top 19 results are selected.

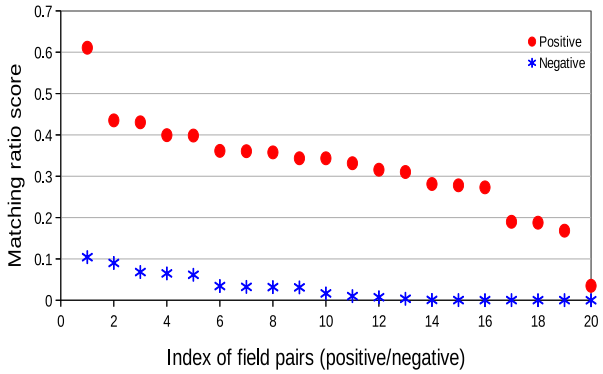

Fig. 10: TPM-based matching ratio score

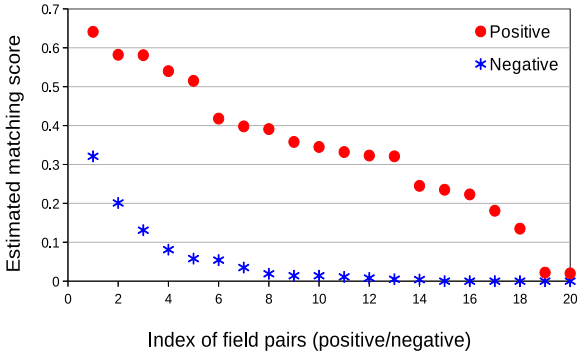

Fig. 11: MH-LSH-based matching score

2) Top-20 similarity results: There are 779 non-numerical fields in the large data sets. Considering that almost all the primary keys in a table are numerical fields, we do not consider primary key constraint matching method for non-numerical matching. The record similarity threshold T_{rn} is set to be 0.4 here. The final top list of matching ratio scores are obtained based on TPM algorithm from all the non-numerical field pairs.

Table VIII shows the top-20 results of field pair matching based on TPM. We can see from that all the field pairs are matched pairs, and they are also confirmed by humans later, which shows the effectiveness of NEMA based on TPM algorithm.

TABLE VIII: Top-20 matching results of non-numerical field pairs

Table.field1	Table.field2	Matching ratio
T_INCI_I2.currentwg_key	T_WK.wkgp_name	0.637
T_INCI_I2.currentwg_key	T_WK.wkgp_desc	0.632
T_INCI.initwg_name	T_WK.wkgp_name	0.63
T_INCI.initwg_name	T_WK.wkgp_desc	0.628
T_WK.wkgmgr_email	T_SUR.eval_email	0.626
T_INCI.creatorwg_name	T_WK.wkgp_name	0.624
T_INCI.creatorwg_name	T_WK.wkgp_desc	0.622
T_INCI.curr_wg_name	T_WK.wkgp_name	0.611
T_INCI.curr_wg_name	T_WK.wkgp_desc	0.607
X_SAH.billto_state	T_SITE.state	0.504
X_SAH.billto_state	T_SITE.cust_state	0.499
T_INCI.initwg_name	T_INCI_I2.curr_wg_name	0.462
T_INCI.initwg_name	T_INCI_I2.wkgrp_name	0.457
T_COT.cpr_country	T_SITE.country	0.453
T_SITE.cust_country	T_COT.cpr_country	0.453
T_INCI_I2.wkgrp_name	T_INCI.curr_wg_name	0.451
T_COT.cpr_country	T_SITE.cust_country	0.447
T_INCI.curr_wg_name	T_INCI_I2.curr_wg_name	0.442
T_SITE.country	T_COT.cpr_country	0.44
T_HW_PROD.erpplatform	X_SCDC.productsub_grp	0.431

E. Comparisons with COMA

We compare our technique NEMA with the COMA system here. COMA is a state of the art and popular hybrid matching tool and system for generic database, XML, etc. It has several versions and the latest version 3.0 supports both schema-level and instance-level matching. Also, it can be evaluated on our network databases compared to other matching tools. We test the numerical and non-numerical ground truth matching of NEMA and COMA in schema-level and instance-level. Their accuracies and differences on matching results are compared and analyzed quantitatively.

1) Comparison of Accuracy: We measure the accuracy and compare the COMA in the schema level and instance level with NEMA for numerical and non-numerical data matching. On the schema level matching, COMA uses the best field matching similarity "0" (which has no corresponding line in the COMA system) as a threshold in the schema-level matching. On the instance level matching, COMA has one similar instance-level matching that uses aggregated maximum record-wise similarities to obtain the final field pair similarities. The record-wise similarity is based on common similarity metrics such as edit distance [37] and trigram [2]. Edit distance is a technique to measure how dissimilar two strings are to one another by counting the minimum number of operations required to transform one string into the other. Trigram is a technique of splitting a string into triples of characters and comparing those to the trigrams of another string. The field matching similarity between two fields A and B in COMA is defined as follows:

$$sim(A,B) = \frac{1}{m+n} \cdot (\sum_{i=1}^{m} max_{j=1\ldots,n}(sim(a_i,b_j)) + \sum_{j=1}^{n} max_{i=1\ldots,m}(sim(b_j,a_i))) \quad (12)$$

Figure 12 shows the accuracy comparisons between COMA and NEMA. COMA-SCH is COMA with schema-level matching algorithm. COMA-ED and COMA-TRG means that

COMA uses edit distance and trigram to measure the record similarity in instance-level matching, respectively. NEMA-(RD+BDP)/TPM is NEMA using RD and BDP in numerical data matching, and TPM in non-numerical data matching. NEMA-MH_LSH indicates that NEMA uses minHash-locality sensitive hashing in non-numerical data matching, without MH-LSH bar shown for numerical matching.

The accuracies of NEMA-(RD+BDP)/TPM in numerical and non-numerical data matching can be up to 95%, as high as COMA's non-numerical data matching, but having higher accuracy than COMA's schema level matching. For numerical matching, the accuracies of COMA-ED and COMA-TRG are 10%-15% lower than NEMA-(RD+BDP)/TPM because of the ineffectiveness to identify non-matched pairs of numerical ground truths. For non-numerical matching, the accuracies of COMA-ED and COMA-TRG are 2% higher than NEMA-(RD+BDP)/TPM. However, the field matching score of COMA is measured based on its general string similarity matching, which is not well applied to the network management database matching for record pair similarity requirements. A large number of pairs with high record similarities in COMA are not thought of as matches in the network management databases, which shows the usefulness of the NEMA non-numerical algorithm.

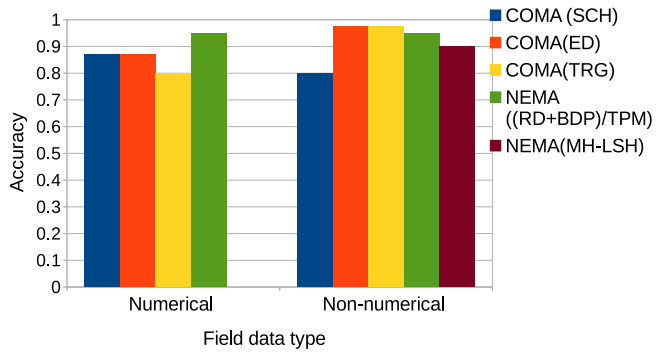

Fig. 12: Accuracy comparison

2) Comparisons of Mismatched Examples: We further analyze the differences of COMA and NEMA in matching the ground truth field pairs. Table IX shows the field pairs in every row and its similarity scores by COMA and NEMA. These field pairs are found to be matched pairs by NEMA with relatively high similarity scores, but COMA shows no similarities with score 0. For COMA, the field names have very few common characters in spelling, even though the semantic commonality exists. NEMA does not rely on the inaccurate schema-level properties, but it uses the record instance for the decisions of field matching, which indirectly considers the semantic correspondences. If the record instances for some matched pairs are incomplete or missing, however, the similarity scores for these field pairs are also low. Table X shows the two field pairs that have low similarities in NEMA. Although the field names in each pair express the same thing semantically, the record instances in the fields are actually incomplete and have very few in common between each other. However, in our databases, the missing or incomplete field pairs are very few compared to the large number of field pairs, which does not affect the overall performance for the numerical matching with NEMA.

TABLE IX: Example of field pairs matched by NEMA, but not by COMA

Table.fieldA	Table.fieldB	COMA similarity	NEMA similarity
T_INCI.ins_site_key	T_SITE.partysite_id	0	0.208
T_OR_HD.creator_id	T_INCI.lastup_by	0	0.643
T_CT.temp_desc	T_PROD.item_desc	0	0.05

TABLE X: Example of field pairs matched by COMA, but not by NEMA

Table.fieldA	Table.fieldB	COMA similarity	NEMA similarity
T_INCI.bl_cot_key	T_CT.bl_cot_key	0.750	0.091
T_SUR.bl_surv_key	T_SUR_ANS.bl_surv_key	0.76	0.009

Here we analyze the specific record pair examples of non-numerical instance-level matching. COMA uses standard edit distance and trigram to calculate the similarities of records, which is not quite suitable for the matching requirement of network management databases. Table XI below shows 9 examples of records pairs and three different kinds of similarities (NEMA record similarity, COMA similarity using edit distance, COMA similarity using trigram). The first 7 rows as one group are thought of as matched record pairs, the last two rows in the other group are non-matched record pairs. We can see from that the similarity of matched pairs based on COMA are quite similar around 0.7 for these two groups, from which is not easy to differentiate. While the similarities by NEMA have good differences (0.333 for non-matched pairs, 0.4 above for matched pairs). This further demonstrates that NEMA is more suitable for the network database matching.

TABLE XI: Examples of record similarity comparisons

Record 1	Record 2	NEMA record similarity	COMA (edit distance)	COMA (trigram)
c900 series	c900 series	1	1	1
c2950 series	c2916 series	0.8	0.833	0.6
1601r series	1601 series	0.775	0.909	0.738
css2950	css2916	0.667	0.714	0.6
C2960	C2960CX	0.577	0.6	0.775
C3560CX	C3560X	0.5	0.833	0.671
AIR35CE	AIR35SE	0.4	0.857	0.6
c800	**s800**	**0.333**	**0.75**	**0.5**
ts 900	**cs900**	**0.333**	**0.8**	**0.667**

3) Comparison of Efficiency: Considering the expensive time consumption for non-numerical field pair matching, we test the efficiency based on the whole non-numerical ground truth. We run the experiment about 20 times on the same machine with the same data and calculate the average computation time without data loading time. Figure 13 shows the total computation time spent for COMA-ED, COMA-TRG, NEMA-TPM and NEMA-MH_LSH. It shows that COMA-ED is the slowest, taking about 12,000 seconds. NEMA-TPM takes about 2500 seconds, which is about 5 times speed-up

than COMA-ED. The fast algorithm NEMA-MH_LSH takes 860 seconds, which is about 14 times faster than COMA-ED. Taking the trade-off of accuracy and efficiency, NEMA based on TPM algorithm is the best choice among these algorithms.

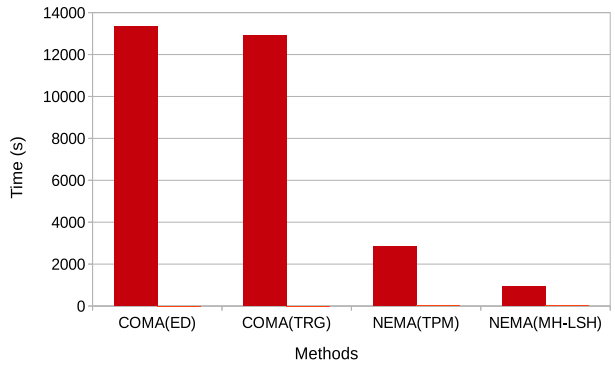

Fig. 13: Efficiency comparison

V. Related Work

The structured data matching is an old and important research topic but unsolved and ever-growing problem, which has a wide range of applications in database integration, migration, semantic query, etc. [6]. In survey papers [35] and [40], the authors proposed a solution taxonomy differentiating between element and structure level, schema and instance level, language and constraint-based matching techniques. Furthermore, P. Shivaiko et.al [40], [41] reviewed the state-of-the-art matching systems which were based on strings, structure, data instance and semantics matching techniques using different schema formats such as database, XML, OWL, RDFS, etc. In database schema applications, previous common matching systems based on schema-level are introduced in several prototypes such as Similarity Flooding (SF) [28], Cupid [27], Artemis [5], Coma [13] etc.

SF [28] is a matching algorithm that models two structured columns to be compared as two directed labeled graphs. It makes use of field data, key properties and the string-based alignment (prefix and suffix test) to obtain the alignments between two nodes of the graph. The similarity is calculated from similar nodes to the adjacent neighbors through propagation. Our NEMA technique only relies on the data instance values to infer the matching of fields, which does not utilize the structured properties and data types. However, SF uses a metric for matching quality based on the intended matching results, which is similar to our accuracy metric based on top-k results.

Cupid [27] is also a generic schema matching algorithm that discovers mappings between schema elements based on their names, data types, constraints, and schema structures with the assistance of a domain specific thesauri. It also explores the tree structures of schema to infer the similarity from upper levels. Similarly, Artemis [5] and DIKE [32] compute the structural affinity for all pairs of classes based on their name affinities and their respective class attributes. Overall, Cupid performs somewhat better than DIKE and Artemis.

Coma [13] is a composite matching system and tool providing extensible library and framework for combining obtained results. It contains mainly 6 elementary matchers using string-based techniques, 5 hybrid matchers using names and structural paths, and one reuse-oriented matcher based on previous matching results . The composite matcher effectively improves the match quality over single matchers using the default combination strategy. Compared to SF, Cupid and DIKE, the overall average matching quality are the best among them [13]. The extended version Coma++ [3] utilizes the shared taxonomy and pivot schema to further improve the overall matching quality. In our evaluation, we compare with the Coma++ method using the default combination strategy and find our technique NEMA overall performans better than COMA in schema-level matching.

Except from the previous matching approaches using field and structural information matching, data instanced-based approaches use the similarity of characteristics of their field instances to determine the similarity of fields. In [26], the authors utilize a corpus that contains schema and mappings between some schema pairs, and learn the constraints from schema statistics to help generate more matching pairs. However, it has to require big human labor to create matching knowledge corpus and to learn from texts, in which our NEMA does not rely on any corpus and dictionary. In [22], [9], the authors uses the mutual information of statistics to measures the similarity of schema instances between two columns to decide the matching, which shows a novel and effective method based on instances. Also, the authors in [34] propose a new sample-driven approach which enables the end-users to easily construct their own data to match the source and target schema. COMA's extended version [16] also proposes two instance-level matching methods based on the constraint of instance data and the content-based matching to measure field matching. The constraint-based method relies on the general, numerical and pattern constraint which has specific limitation to the specific data which is not suitable for the network databases. The content-based matching depends on the aggregation of similarity scores of instance contents and it is kind of similar to our NEMA technique on content-based similarity measurement, which are compared with NEMA in the experimental evaluations.

To sum up, most of currently popular matching approaches and systems focus on schema-level information matching. The data instances level matching approaches using field record values are mostly based on some statistical models and machine learning from corpus. We further explore the database instance matching by comparing the field value using different metrics and propose effective and overall matching algorithms considering the characteristic for network database matching specifically for a graph database construction for efficient data query, analysis and management.

VI. Conclusion

In this paper we propose a systematic technique NEMA to match databases for network management. Different from previous database matching approaches, we design a technique

to match numerical and non-numerical fields in instance-level respectively, which can effectively be integrated into a graph database for network management and analysis. For numerical matching, we propose range difference similarity and bucket dot product similarity metrics. For non-numerical matching, we design record-wise similarity matching algorithm based on top priority match metric and also propose applying minHash-locality sensitive hashing algorithm, which reduces the matching time for large databases. The results of NEMA are demonstrated to be promising with better efficiency and comparable accuracy than the baseline algorithms.

With the explosion of big data and popularity of distributed graph processing systems, this work has the potential to significantly reduce the human work involving identifying the matching fields for a large graph database construction and also be applied for large-scale data matching. A majority of partial matching pairs can be found by our matching algorithms which are not easily detected by humans.

REFERENCES

[1] C. C. Aggarwal and H. Wang. Graph data management and mining: A survey of algorithms and applications. In *Managing and mining graph data*, pages 13–68. Springer, 2010.

[2] R. C. Angell, G. E. Freund, and P. Willett. Automatic spelling correction using a trigram similarity measure. *Information Processing & Management*, 19(4):255–261, 1983.

[3] D. Aumueller, H.-H. Do, S. Massmann, and E. Rahm. Schema and ontology matching with coma++. In *Proceedings of the 2005 ACM SIGMOD international conference on Management of data*, pages 906–908. ACM, 2005.

[4] A. Bhattacharjee and H. Jamil. Ontomatch: A monotonically improving schema matching system for autonomous data integration. In *Information Reuse & Integration, 2009. IRI'09. IEEE International Conference on*, pages 318–323. IEEE, 2009.

[5] S. Castano and V. De Antonellis. Global viewing of heterogeneous data sources. *Knowledge and Data Engineering, IEEE Transactions on*, 13(2):277–297, 2001.

[6] S. Castano, A. Ferrara, and S. Montanelli. Matching techniques for data integration and exploration: From databases to big data. In *A Comprehensive Guide Through the Italian Database Research Over the Last 25 Years*, pages 61–76. Springer, 2018.

[7] M. Cheatham and P. Hitzler. String similarity metrics for ontology alignment. In *The Semantic Web–ISWC 2013*, pages 294–309. Springer, 2013.

[8] C. Chen, B. Golshan, A. Y. Halevy, W.-C. Tan, and A. Doan. Biggorilla: An open-source ecosystem for data preparation and integration. *IEEE Data Eng. Bull.*, 41(2):10–22, 2018.

[9] W. Chen, H. Guo, F. Zhang, X. Pu, and X. Liu. Mining schema matching between heterogeneous databases. In *Consumer Electronics, Communications and Networks (CECNet), 2012 2nd International Conference on*, pages 1128–1131. IEEE, 2012.

[10] C. Coronel and S. Morris. *Database systems: design, implementation, & management*. Cengage Learning, 2016.

[11] C. Daraio, M. Lenzerini, C. Leporelli, P. Naggar, A. Bonaccorsi, and A. Bartolucci. The advantages of an ontology-based data management approach: openness, interoperability and data quality. *Scientometrics*, pages 1–15, 2016.

[12] M. Dastbaz, C. Pattinson, and B. Akhgar. *Green information technology: A sustainable approach*. Morgan Kaufmann, 2015.

[13] H.-H. Do, S. Melnik, and E. Rahm. Comparison of schema matching evaluations. In *Net. ObjectDays: International Conference on Object-Oriented and Internet-Based Technologies, Concepts, and Applications for a Networked World*, pages 221–237. Springer, 2002.

[14] H.-H. Do and E. Rahm. Coma: a system for flexible combination of schema matching approaches. In *Proceedings of the 28th international conference on Very Large Data Bases*, pages 610–621. VLDB Endowment, 2002.

[15] X. L. Dong and D. Srivastava. Big data integration. *Synthesis Lectures on Data Management*, 7(1):1–198, 2015.

[16] D. Engmann and S. Massmann. Instance matching with coma+. 2007.

[17] D. Gomez-Cabrero, I. Abugessaisa, D. Maier, A. Teschendorff, M. Merkenschlager, A. Gisel, E. Ballestar, E. Bongcam-Rudloff, A. Conesa, and J. Tegnér. Data integration in the era of omics: current and future challenges, 2014.

[18] B. Gu, Z. Li, X. Zhang, A. Liu, G. Liu, K. Zheng, L. Zhao, and X. Zhou. The interaction between schema matching and record matching in data integration. *IEEE Transactions on Knowledge and Data Engineering*, 29(1):186–199, 2016.

[19] D. S. Himmelstein and S. E. Baranzini. Heterogeneous network edge prediction: a data integration approach to prioritize disease-associated genes. *PLoS computational biology*, 11(7):e1004259, 2015.

[20] E. Ioannou and Y. Velegrakis. Embench++: Data for a thorough benchmarking of matching-related methods. *Semantic Web*, (Preprint):1–16, 2019.

[21] D. Jang, T. Kim, and H. Kim. History management for network information of iot devices. In *International Conference on Security with Intelligent Computing and Big-data Services*, pages 138–145. Springer, 2017.

[22] J. Kang and J. F. Naughton. On schema matching with opaque column names and data values. In *Proceedings of the 2003 ACM SIGMOD international conference on Management of data*, pages 205–216. ACM, 2003.

[23] J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of massive datasets*. Cambridge University Press, 2014.

[24] M. Lissandrini, M. Brugnara, and Y. Velegrakis. Beyond macrobenchmarks: Microbenchmark-based graph database evaluation. *Proceedings of the VLDB Endowment*, 12(4):390–403, 2018.

[25] T. Lose, P. van Heusden, and A. Christoffels. Combat-tb-neodb: fostering tuberculosis research through integrative analysis using graph database technologies. *Bioinformatics*, 2019.

[26] J. Madhavan, P. A. Bernstein, A. Doan, and A. Halevy. Corpus-based schema matching. In *21st International Conference on Data Engineering (ICDE'05)*, pages 57–68. IEEE, 2005.

[27] J. Madhavan, P. A. Bernstein, and E. Rahm. Generic schema matching with cupid. In *VLDB*, volume 1, pages 49–58, 2001.

[28] S. Melnik, H. Garcia-Molina, and E. Rahm. Similarity flooding: A versatile graph matching algorithm and its application to schema matching. In *Data Engineering, 2002. Proceedings. 18th International Conference on*, pages 117–128. IEEE, 2002.

[29] J. J. Miller. Graph database applications and concepts with neo4j. In *Proceedings of the Southern Association for Information Systems Conference, Atlanta, GA, USA*, volume 2324, 2013.

[30] J. K. Nguyen, S. W. Kurth, et al. Contextual graph matching based anomaly detection, June 14 2016. US Patent 9,367,809.

[31] H. Nottelmann and U. Straccia. Information retrieval and machine learning for probabilistic schema matching. *Information processing & management*, 43(3):552–576, 2007.

[32] L. Palopoli, G. Terracina, and D. Ursino. The system dike: Towards the semi-automatic synthesis of cooperative information systems and data warehouses. In *ADBIS-DASFAA Symposium*, pages 108–117, 2000.

[33] G. Pantuza, F. Sampaio, L. F. Vieira, D. Guedes, and M. A. Vieira. Network management through graphs in software defined networks. In *10th International Conference on Network and Service Management (CNSM) and Workshop*, pages 400–405. IEEE, 2014.

[34] L. Qian, M. J. Cafarella, and H. Jagadish. Sample-driven schema mapping. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pages 73–84. ACM, 2012.

[35] E. Rahm and P. A. Bernstein. A survey of approaches to automatic schema matching. *the VLDB Journal*, 10(4):334–350, 2001.

[36] A. D. Raut and M. Atique. A survey of indexing techniques for xml database. *Compusoft*, 3(1):461, 2014.

[37] E. S. Ristad and P. N. Yianilos. Learning string-edit distance. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(5):522–532, 1998.

[38] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proceedings of the VLDB Endowment*, 11(4):420–431, 2017.

[39] C. Shi, Y. Li, J. Zhang, Y. Sun, and S. Y. Philip. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, 29(1):17–37, 2016.

[40] P. Shvaiko and J. Euzenat. A survey of schema-based matching approaches. In *Journal on data semantics IV*, pages 146–171. Springer, 2005.

[41] P. Shvaiko and J. Euzenat. Ontology matching: state of the art and future challenges. *IEEE Transactions on knowledge and data engineering*, 25(1):158–176, 2013.

[42] M. Stonebraker and I. F. Ilyas. Data integration: The current status and the way forward. *IEEE Data Eng. Bull.*, 41(2):3–9, 2018.
[43] N. Sünderhauf, T. T. Pham, Y. Latif, M. Milford, and I. Reid. Meaningful maps with object-oriented semantic mapping. In *Intelligent Robots and Systems (IROS), 2017 IEEE/RSJ International Conference on*, pages 5079–5085. IEEE, 2017.
[44] J. Wang, Y. Wu, N. Yen, S. Guo, and Z. Cheng. Big data analytics for emergency communication networks: A survey. *IEEE Communications Surveys & Tutorials*, 18(3):1758–1778, 2016.

Fubao Wu received her Bachelor degree in Electronic Information Engineering from Northeastern University, China in 2008 and Master of Engineering in Electronic Science and Technology from University of Science and Technology of China in 2011. He is currently pursuing his Ph.D. degree in Electrical and Computer Engineering at University of Massachusetts, Amherst. His research interests are data analytics, graph analytics and video analytics.

Han Hee Song was Lead Data Scientist at Cisco Systems. He was Principal Data Scientist at Symantec Corp. and Senior Member of Technical Sta at CTO office of Narus, Inc. His research focuses on privacy analysis of mobile users and protective measures for cyber terrorism. Dr. Song received a Ph.D. and M.A. in Computer Science from the University of Texas at Austin in 2011 and 2006, respectively. He received a B.S. from Yonsei University, Seoul, Korea in 2004.

Jiangtao Yin received the Ph.D. degree in Electrical and Computer Engineering from the University of Massachusetts at Amherst, in 2016. He is currently working at Palo Alto Networks. His current research interests include cloud computing, large-scale data processing, stream processing, and distributed systems.

Lixin Gao received the Ph.D. degree in computer science from the University of Massachusetts at Amherst, in 1996. Now she is a professor of electrical and computer engineering with the University of Massachusetts at Amherst. Her research interests include social networks, Internet routing, network virtualization and cloud computing. Between May 1999 and January 2000, she was a visiting researcher in ATT Research Labs and DIMACS. She was an Alfred P. Sloan fellow between 2003-2005 and received an NSF CAREER Award in 1999. She won the best paper award from IEEE INFOCOM 2010 and ACM SoCC 2011, and the test-of-time award in ACM SIGMETRICS 2010. She received the Chancellors Award for Outstanding Accomplishment in Research and Creative Activity in 2010. She is a fellow of the IEEE and ACM.

Mario Baldi is Director of Technology at Cisco Systems and Associate Professor at Politecnico di Torino. He was Data Scientist Director at Symantec Corp., Inc., Principal Member of Technical Sta at Narus, Inc., Principal Architect at Embrane, Inc.; Vice Dean of the PoliTong Sino-Italian Campus at Tongji University, Shanghai; Vice President for Protocol Architecture at Synchrodyne Networks, Inc., New York. Through his research, teaching and professional activities, Mario Baldi has built considerable knowledge and expertise in big data analytics, next generation network data analysis, internetworking, high performance switching, optical networking, quality of service, multimedia networking, trust in distributed software execution, and computer networks in general.

Narendra Anand worked as a Research Scientist at Cisco Systems as a part of the Forward-looking Analytics, Software, and Technolgies (FAST) program. He received his Ph.D. degree in May of 2015 at Rice University in the Department of Electrical and Computer Engineering. His research focuses on the design and implementation of novel, cross-layer, Multi-user Multiple-Input-Multiple-Output (MU-MIMO) communication protocols.